Employee Payroll and Statutory Deduction Management System

Final Project Report - DBMS Project 36

Student: G MADHAN (25WU0102344), AI ML RHINOS

> All data is synthetic. Deduction rates are illustrative, not official statutory rates. Bank transfers are simulated.

1. Introduction and problem statement

Small and mid-sized organisations often calculate salaries in spreadsheets. This causes calculation errors (loss of pay, statutory deductions, loan recovery), inconsistent records, and no reliable payslip history or audit trail. Project 36 replaces that with a relational database as the single source of truth and a web application on top of it.

2. Objectives (from Review 1) and outcome

All twelve functional requirements from Review 1 are implemented: employee records; departments and designations; salary structures; attendance summaries; earnings; statutory and other deductions; loans and repayments; payroll periods; gross / deductions / net calculation; payslips; bank transfer details (simulated); payroll reports. Additions made during implementation: `loan_repayment` (repayment history), `payroll_config` (editable illustrative rates), and - in the final stage - login with roles (`app_user`), database triggers, an audit trail (`audit_log`), reporting views and a Database Explorer.

3. System architecture

Browser -> Flask routes (`app/__init__.py`, with a `before_request` guard for login, roles and CSRF) -> generic CRUD views driven by a configuration dictionary (`app/crud.py`), payroll engine (`app/payroll.py`), reports (`app/reports.py`), database explorer (`app/dbinfo.py`), authentication helpers (`app/auth.py`) -> `app/db.py` (thin layer, ? placeholders converted for MySQL, driver differences hidden) -> SQLite (default) or MySQL 8. Front end: Jinja2, Bootstrap 5, Chart.js, all vendored (works offline).

4. Database design

16 tables: department, designation, payroll_config, employee, salary_structure, payroll_period, attendance_summary, earnings, deductions, loan, payroll_detail, loan_repayment, payslip, bank_transfer, app_user, audit_log. **Constraints:** 16 primary keys, 17 foreign keys, 14 UNIQUE, 23 CHECK, 8 indexes. **Views:** v_payroll_summary, v_department_cost, v_loan_ledger. **Triggers:** 6. **Relationships:** see `docs/ER_DIAGRAM.md` (Mermaid diagram). Department / Designation 1:N Employee; Employee 1:1 Salary Structure; Employee 1:N Attendance / Earnings / Deductions / Loan / Payroll Detail; Payroll Period 1:N Payroll Detail; Payroll Detail 1:1 Payslip and 1:1 Bank Transfer; Loan 1:N Loan Repayment; Employee 1:0..1 App User. **Normalisation:** 3NF. `payroll_detail` is a deliberate results snapshot so a payslip never changes after processing; loss-of-pay days are derived (working - present - paid leave), not stored.

Referential actions: ON DELETE RESTRICT protects paid history and login accounts; CASCADE removes employee-owned inputs. **Scripts:** `database/mysql/schema.sql` (master DDL), `database/sqlite/schema.sql` (generated), `seed.sql` (synthetic: 30 employees, 6 departments, 8 designations, 6 loans), `views.sql`, `mysql|sqlite/triggers.sql`, `queries.sql`.

4.1 Triggers (database-level rules)

Trigger	When	Effect
trg_payroll_detail_lock_insert / _update / _delete	before change to `payroll_detail`	refuses the change when the payroll period is not OPEN ("Payroll history is locked")
trg_salary_update_audit	after salary_structure update	writes old / new salary components to `audit_log`
trg_employee_status_audit	after employee update	records ACTIVE / INACTIVE changes
trg_config_audit	after payroll_config update	records rate changes

SQLite uses `RAISE(ABORT, ...)`; MySQL uses `SIGNAL SQLSTATE '45000'`, so the two trigger files are written by hand. The application also locks inputs of processed periods, so the database rule is a second line of defence.

5. Payroll calculation

See `docs/ASSUMPTIONS.md`. Worked example (EMP003, October 2026): fixed gross 1,00,130; PF 1,800; PT 200; TDS 2,506.50; loan EMI 5,000; total deductions 9,506.50; net 90,623.50. Processing is one database transaction: every active employee is calculated or the whole run is rolled back; a period is processed only once (OPEN -> PROCESSED -> PAID).

6. Security and access control

Login with three roles (ADMIN, HR, EMPLOYEE); passwords stored only as salted PBKDF2-SHA256 hashes; CSRF token on every POST; per-username login throttling (5 failures / 5 minutes, in memory); a new session on login; an employee can open only their own payslips; the last active administrator cannot be deactivated. The Database Explorer's SQL console accepts one SELECT, runs on a read-only connection with a 5-second limit and a 200-row limit, and refuses the `app_user` table. Scope: a local demo - not hardened for the internet (no HTTPS; the forgot-password link is displayed on screen because there is no e-mail server - it is signed, expires after 15 minutes and works only once).

7. SQL operations

`database/queries.sql` demonstrates CRUD; INNER and LEFT JOINs including a 5-table payslip join; aggregates with GROUP BY and HAVING; scalar, IN, NOT EXISTS, correlated and derived-table subqueries; consistency checks (net = gross - deductions; itemised deductions equal totals); the three views; a self-cleaning trigger / audit demonstration; and integrity-failure demonstrations (commented out because they are expected to fail). The file runs completely on both SQLite and MySQL 8 and is executed by the test suites.

8. Implementation summary

Pages: sign-in, dashboard, employees (+profile), departments, designations, salary structures, attendance, earnings, deductions, loans (+repayments), payroll periods, payroll register (+CSV), payslips, bank transfers, 7 reports (each shows its SQL), rates & settings, users & roles, database explorer, my payslips (employee), change password. Results from the demo database (synthetic): August 2026 net 18,36,259; September 18,06,179; October 18,18,956; 29 employees paid each month (one inactive employee is skipped).

9. Testing

74 automated tests on SQLite (calculation 5, database 9, payroll processing 9, triggers and views 6, authentication and roles 13, password reset 7, amount in words 2, explorer 6, web 17) - all pass; 11 integration tests on a real MySQL 8.0.46 server - all pass. See `docs/TESTING.md`, `tests/test_results.txt`, `tests/test_results_mysql.txt`. Screenshots were captured from the running application with headless Chromium. The MySQL run found and fixed one real bug: MySQL reports CHECK violations (error 3819) and trigger signals (1644) as `OperationalError`; they are now handled as integrity errors on both engines.

10. Challenges and solutions

Challenge	Solution
MySQL and SQLite differ in syntax	MySQL schema is the master; the SQLite schema is generated; portable SQL; trigger files written per dialect
Triggers contain semicolons	`--  @@` block markers split the MySQL trigger script; SQLite runs it whole
MySQL refuses trigger creation with binary logging	`log_bin_trust_function_creators` documented; init script explains error 1419
MySQL reports CHECK / SIGNAL as OperationalError	mapped to integrity errors in the DB layer
Exact money arithmetic	Decimal with 2-place rounding; Decimal converted for SQLite
Edits after payroll would corrupt history	Period status locks inputs, triggers lock `payroll_detail`, processing is one transaction
Safe ad-hoc SQL for the admin	read-only connection + SELECT-only filter + timeout + hidden hashes

11. Limitations

Bank transfers are simulated; rates are illustrative; local demo server without HTTPS; in-memory login throttle; audit rows do not store the web user's name (triggers cannot see it); small synthetic dataset; MySQL tested only on 8.0.46; the GitHub Actions workflow has not yet run; only headless Chromium tested.

12. Future work

Official tax and statutory slabs; PDF / e-mail payslips; audit entries with the acting user; bank bulk-payment file export; deployment behind HTTPS; role-specific dashboards.

13. Conclusion

The project turns the Review 1 design into a working, tested payroll system in which the database itself enforces the rules (constraints, triggers, views) and the application adds secure access. It demonstrates normalisation, keys, constraints, transactions, triggers, views, joins, subqueries and aggregation, but is not a production payroll system.

14. References

Flask, SQLite, MySQL 8, PyMySQL, Bootstrap 5, Chart.js documentation; R. Elmasri and S. Navathe, *Fundamentals of Database Systems*; A. Silberschatz, H. Korth and S. Sudarshan, *Database System Concepts*; Project 36 Review 1 presentation.